

INFORME DETALLADO DE FUNCIONAMIENTO

Biblioteca Digital de Partituras EMMFAB

1. VISION GENERAL

La aplicacion es una biblioteca digital de partituras orientada a instrumentos de viento (trompeta y baritono) y ritmos folkloricos bolivianos (Morenada, Diablada, Tinkus, etc.). Permite a usuarios autenticados visualizar, filtrar y descargar partituras en PDF. El administrador puede gestionar ritmos y partituras (agregar, editar, eliminar) y ademas componer nuevas partituras mediante notacion ABC.

Arquitectura cliente-servidor con Node.js/Express en el backend, EJS para vistas dinamicas y Tailwind CSS para disenio responsive. Almacenamiento local basado en archivos JSON, con carpetas para los PDFs subidos. En entornos serverless como Vercel el almacenamiento de archivos es efimero, por lo que se recomienda migrar a servicios en la nube para produccion real.

2. ESTRUCTURA DEL PROYECTO

biblioteca-partituras/

```
├── app.js          # Servidor Express, rutas, logica principal
├── package.json    # Dependencias y scripts
├── data/           # Almacenamiento JSON (usuarios, ritmos, partituras)
│   ├── users.json
│   ├── rhythms.json
│   └── scores.json
├── uploads/        # PDFs subidos (solo local)
├── public/         # Archivos estaticos (imagenes, CSS)
│   ├── img/        # Logos, fondos, cabeceras
│   └── style.css (opcional)
└── views/          # Plantillas EJS
    ├── login.ejs
    ├── index.ejs
    ├── admin.ejs
    └── compose.ejs
```

3. COMPONENTES DEL BACKEND (app.js)

3.1 Configuración inicial

- Express con middleware para datos URL-encoded, archivos estáticos (public) y manejo de sesiones (express-session).
- Multer configura almacenamiento en disco (para entornos locales) y filtro de tipo de archivo (solo PDF).

3.2 Gestión de datos (JSON)

- Funciones `safeReadJSON` y `writeJSON` para leer/escribir archivos de manera segura (si el archivo no existe o está vacío, lo crea con datos por defecto).
- Archivos:
 - `users.json`: almacena usuarios con `id`, `username`, `password` (sin hash) y `role` (user o admin). Usuario demo: EMMFAB/2026, admin: admin/admin123.
 - `rhythms.json`: lista de ritmos con `id` y `name`. Incluye 34 ritmos predefinidos (desde Morenada hasta Marcha fúnebre).
 - `scores.json`: lista de partituras con `id`, `title`, `filename` (ruta del PDF en uploads/), `rhythm_id`, `instrument` (trompeta/baritono/ambos) y `upload_date`.

3.3 Autenticación y autorización

- Middleware `requireLogin`: verifica que exista `req.session.user`, de lo contrario redirige a `/login`.
- Middleware `requireAdmin`: verifica que el rol del usuario sea admin, de lo contrario retorna 403.
- Las sesiones se guardan en memoria (en producción se recomienda usar una store externa como Redis).

3.4 Rutas principales

- GET `/login` : Muestra formulario de login.
- POST `/login` : Valida credenciales contra `users.json`, crea sesión y redirige a `/`.
- GET `/logout` : Destruye sesión y redirige a `/login`.
- GET `/` (`requireLogin`) : Página principal. Lee ritmos y partituras, filtra por `rhythm` e `instrument` según parámetros `?rhythm=...&instrument=....`. Renderiza `index.ejs`.

- GET /download/:id (requireLogin) : Busca la partitura en scores.json, obtiene el nombre del archivo y lo envia como descarga (res.download).
- GET /admin (requireLogin, requireAdmin) : Panel de administracion: muestra formularios y listados. Renderiza admin.ejs.
- POST /admin/add-rhythm (requireAdmin) : Agrega un nuevo ritmo al JSON.
- POST /admin/edit-rhythm/:id (requireAdmin) : Edita el nombre de un ritmo existente.
- POST /admin/delete-rhythm/:id (requireAdmin) : Elimina un ritmo solo si no tiene partituras asociadas.
- POST /admin/upload-score (requireAdmin, upload.single('pdf')) : Sube un PDF, asigna titulo, ritmo e instrumento, crea entrada en scores.json.
- POST /admin/edit-score/:id (requireAdmin) : Actualiza titulo, ritmo o instrumento de una partitura.
- POST /admin/delete-score/:id (requireAdmin) : Elimina la partitura (borra el archivo fisico y el registro JSON).
- GET /compose (requireAdmin) : Muestra el editor de notacion ABC (vista compose.ejs).

4. COMPONENTES DEL FRONTEND

4.1 Login (login.ejs)

- Fondo con imagen del Carnaval de Oruro (superposicion oscura).
- Formulario con campos usuario/contraseña.
- Logo circular (se intenta cargar FABLOGO.jpeg; si falla, se muestra un placeholder).
- Mensajes de error en caso de credenciales invalidas.

4.2 Pagina principal (index.ejs)

- Barra de navegacion: muestra logo, nombre, enlace a admin (si corresponde) y boton de salir.
- Encabezado: imagen del carnaval con texto superpuesto.
- Mensaje de bienvenida (para que no se vea vacio).
- Filtros:
 - Por ritmo: se muestran todos los ritmos del JSON mediante botones que envian formulario GET (evita problemas de enrutamiento).
 - Por especialidad (trompeta/baritono/todos): mediante enlaces con parametros.

- Listado de partituras: cada tarjeta muestra titulo, ritmo, instrumento, fecha y botones:
 - Descargar PDF: enlace directo a /download/:id.
 - Vista previa: abre un modal con un iframe que muestra el PDF.
 - (Solo admin) Editar y Eliminar: abren modales o confirmacion.
- Contador de resultados encontrados.
- Modal de vista previa: incrusta el PDF con un iframe; permite descargar desde el mismo modal.
- Modales de edicion: formularios dinamicos para editar partituras (titulo, ritmo, instrumento).
- Formulario oculto de eliminacion: para envio POST sin recarga de pagina (con confirmacion).

4.3 Panel de administracion (admin.ejs)

- Seccion para agregar ritmo: campo de texto y boton.
- Seccion para subir PDF: formulario con titulo, selector de ritmo (dinamico), selector de instrumento y carga de archivo.
- Lista de ritmos existentes: cada ritmo muestra botones editar y eliminar. Los modales de edicion de ritmo se abren con JavaScript.
- Tabla de partituras: muestra todas las partituras con sus datos. Incluye botones de editar y eliminar (con confirmacion). La edicion abre un modal con campos de titulo, ritmo e instrumento.

4.4 Compositor de partituras (compose.ejs)

- Utiliza la libreria abcjs para renderizar notacion musical a SVG.
- Area de texto para escribir codigo ABC (ejemplo de Morenada).
- Boton para actualizar la vista previa.
- Boton para exportar a PDF: mediante html2canvas + jspdf, captura el SVG generado y produce un PDF descargable.
- El PDF generado debe ser subido manualmente por el administrador en el panel normal.

5. FLUJO DE DATOS Y PROCESOS

5.1 Autenticacion

- Usuario envia formulario /login.
- Servidor lee users.json, busca coincidencia.
- Si existe, guarda session.user y redirige a /.
- En cada peticion protegida, se verifica la sesion.

5.2 Filtrado de partituras

- Pagina principal recibe parametros ?rhythm=...&instrument=...
- Se cargan todos los ritmos y partituras.
- Se aplica filtro por ritmo (coincidencia de nombre -> rhythm_id).
- Se aplica filtro por instrumento: si el instrumento elegido es trompeta, solo se muestran partituras con instrument igual a trompeta o ambos. Igual para baritono.
- Se renderiza la vista con las partituras filtradas.

5.3 Subida de partituras (solo admin)

- Formulario multipart envia titulo, ritmo, instrumento y archivo PDF.
- Multer guarda el archivo en uploads/ con nombre unico (timestamp + originalname).
- Se lee scores.json, se genera un nuevo ID, se agrega el registro y se escribe de nuevo el archivo.
- Se redirige a /admin con mensaje de exito.

5.4 Vista previa del PDF

- Al hacer clic en "Vista previa", se pasa la URL de descarga a un iframe dentro de un modal. El navegador renderiza el PDF automaticamente (si tiene soporte). El usuario puede verlo sin descargar.

5.5 Edicion/Eliminacion

- Edicion de ritmo: cambia el nombre en rhythms.json. Si hay partituras asociadas, el ritmo no se elimina.
- Eliminacion de ritmo: primero verifica que no haya partituras con ese rhythm_id. Si existe, deniega.
- Edicion de partitura: actualiza titulo, rhythm_id e instrument en scores.json.

- Eliminacion de partitura: ademas de borrar el registro, elimina el archivo PDF fisico de uploads/.

6. SEGURIDAD Y CONSIDERACIONES

- Contraseñas en texto plano: en un entorno real se debe usar hashing (bcrypt).
- Sesiones en memoria: no escalable para multiples servidores; usar Redis o similar en produccion.
- Validacion de archivos: Multer filtra por MIME type application/pdf, pero se recomienda adicionalmente escanear malware.
- Inyeccion de codigo: los parametros se escapan adecuadamente en las vistas (EJS escapa HTML por defecto) y en las consultas a JSON no hay inyeccion SQL.
- CSRF: no implementado. Para formularios sensibles (subida, eliminacion) se recomienda anadir tokens.
- Acceso a rutas: las rutas de administracion estan protegidas por requireAdmin.
- Almacenamiento local: en entornos serverless (Vercel) el sistema de archivos es efimero y de solo lectura para datos persistentes. Por tanto, las subidas de PDF y las modificaciones en JSON no persisten entre ejecuciones. Para funcionar en Vercel se requiere migrar a almacenamiento en la nube (Cloudinary, S3) y a una base de datos (MongoDB Atlas, Supabase).

7. EXPERIENCIA DE USUARIO (UI/UX)

- Responsive: Tailwind CSS garantiza adaptacion a movil, tablet y desktop (clases flex-wrap, grid-cols-1 md:grid-cols-2, etc.).
- Estetica: colores purpura, imagenes del Carnaval de Oruro, tipografia moderna.
- Interactividad: modales para edicion y vista previa, confirmaciones de eliminacion (JavaScript nativo), botones con transiciones hover.
- Accesibilidad: se usan etiquetas semanticas y atributos alt en imagenes (aunque pueden mejorarse).
- Manejo de errores: mensajes flash en la URL para exito/error en el panel de administracion.

8. DESPLIEGUE Y OPERACION

Local (desarrollo)

- Instalar Node.js.
- npm install en la raiz.
- node app.js.
- Acceder a <http://localhost:3000>.

En Vercel (produccion limitada)

- Se debe modificar app.js para exportar la aplicacion (`module.exports = app`) en lugar de llamar a `app.listen()` incondicionalmente.
- Crear `vercel.json` que redirija todas las rutas a app.js y sirva archivos estaticos.
- Problema critico: el almacenamiento de archivos en `uploads/` y `data/` no es persistente. Cada vez que la funcion serverless se reinicia (o se despliega una nueva version), los datos se pierden.
- Solucion: integrar Cloudinary para PDFs y MongoDB/Supabase para los JSON.

Recomendaciones para produccion real

- Reemplazar `multer.diskStorage` por `multer.memoryStorage()` y subir los buffers a Cloudinary.
- Reemplazar las lecturas/escrituras de archivos JSON por operaciones en una base de datos.
- Utilizar variables de entorno para credenciales.
- Implementar autenticacion robusta con JWT y hashing de contraseñas.

9. POSIBLES MEJORAS FUTURAS

- Sistema de busqueda por texto en titulos.
- Categorizacion por instrumento mas granular (trompeta en Do, trompeta en Si bemol, baritono en Do, etc.).
- Reproductor de audio integrado (midi o mp3) para escuchar la pieza.
- Editor de partituras visual (drag & drop de notas) en lugar de solo notacion ABC.
- Perfiles de usuario con listas de favoritos.
- Estadisticas (numero de descargas por partitura).
- Exportacion a otros formatos (MusicXML, MIDI).

10. CONCLUSION

La aplicacion cumple con todos los requisitos funcionales establecidos: autenticacion diferenciada, gestion de ritmos y partituras, filtros, subida/descarga de PDFs, disen

atractivo y responsive. El código está estructurado de forma comprensible y extensible. Sin embargo, para un despliegue en producción real (especialmente en plataformas serverless como Vercel), es obligatorio reemplazar el almacenamiento local por servicios externos. El informe presentado detalla cada aspecto de su funcionamiento, desde la inicialización hasta la interacción del usuario, pasando por la lógica de negocio y las decisiones de implementación.